



UNIVERSITÉ ÉVRY
PARIS-SACLAY

TP VPN

OpenVPN vs Wireguard

M1 Computer & Network Systems

Nom: KOKPATA Eudes
Enseignant: Mehdi DENOUE

2025-2026

Table des Matières

Introduction.....	3
Architecture.....	4
Partie 1 : Préparation de l'architecture et Routage.....	5
1.1 Configuration des machines.....	5
1.2 Explication du routage:.....	7
1.3 Test des pings.....	8
Partie 2 : Mise en place d'OpenVPN.....	10
2.1 Configuration VPN.....	11
Partie 3 : Étude du routage global.....	15
3.1 Configuration du VPN pour tous les paquets sortant du C1.....	15
3.2 Explication du Table de routage C1.....	16
Partie 4 : L'infrastructure DNS.....	18
4.1 Étape 1 : S3 en serveur DNS Cache (Bind9).....	18
4.2 Étape 2 : Le Serveur DNS Interne (sur S5).....	19
4.3 Étape 3 : Test de ssh de C1.....	20
Configuration finale du serveur VPN (S4).....	21
Test de résilience (Coupure de l'interface physique) :.....	23
WireGuard.....	25
Architecture.....	25
Q1.....	25
Configuration des machines:.....	26
Test de ping VPN:.....	27
Q2.....	28
Redirection du trafic (AllowedIPs = 0.0.0.0/0).....	28
Q3.....	29
Rétablissement du tunnel (Stateless vs Stateful).....	30
Q4.....	31
Q5.....	31
Comparaison des fichiers de configurations finaux des serveurs:.....	32
Conclusion.....	33
Ressources.....	34

Introduction

Au cœur des Tunnels Sécurisés

Imaginez que vous deviez envoyer un document ultra-secret à un collègue de l'autre côté du pays, mais que la seule route disponible soit publique, bondée et surveillée de toutes parts (Internet). Comment faire pour que personne ne lise votre message en chemin ? La solution est de construire un tunnel blindé, invisible et privé, directement entre vous et votre collègue.

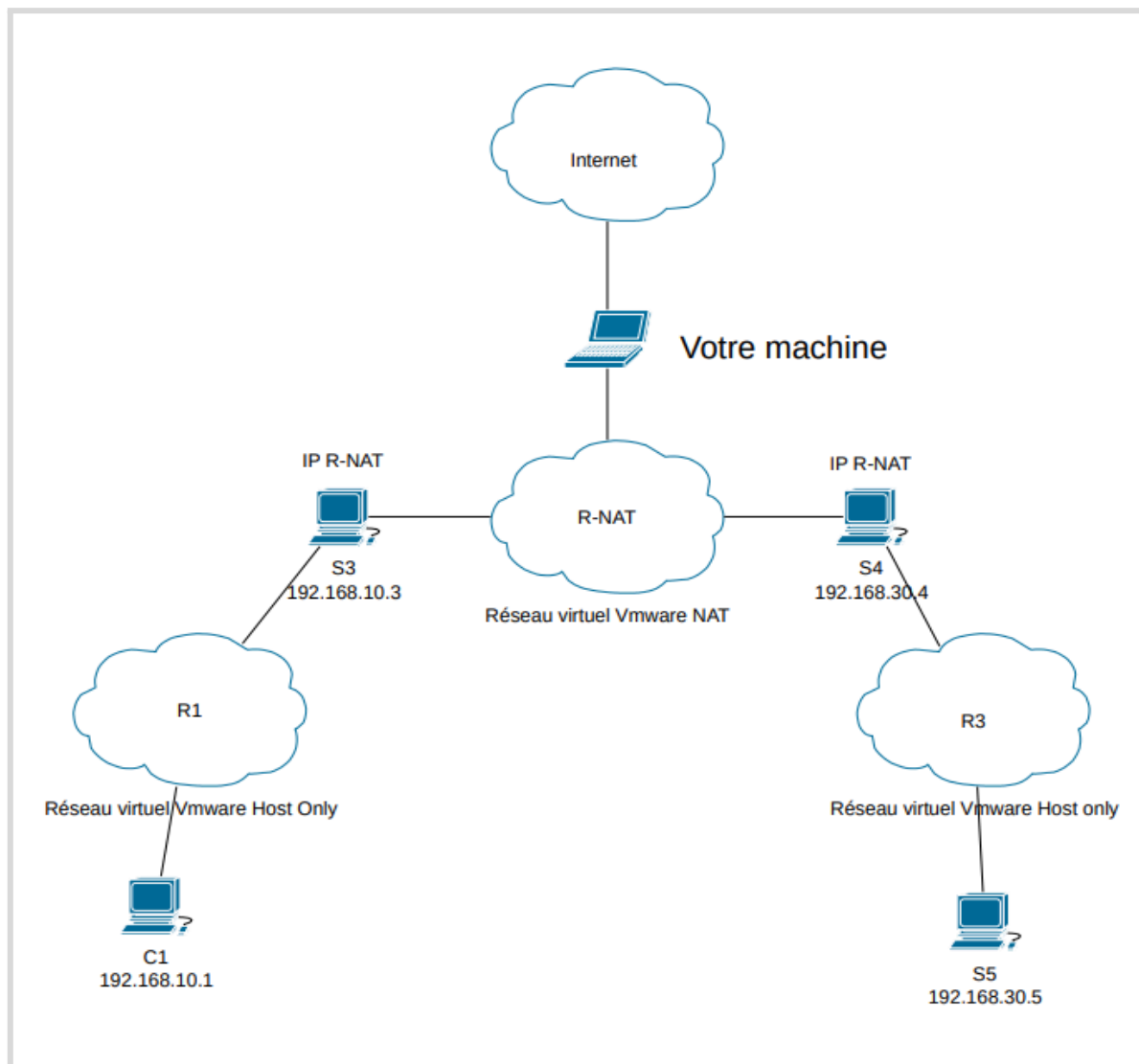
C'est exactement le rôle d'un **VPN** (*Virtual Private Network*).

Dans ce TP, nous allons plonger dans les coulisses de la sécurité réseau en construisant et en connectant notre propre infrastructure. Pour cela, nous allons mettre en place et confronter deux technologies incontournables :

- **OpenVPN (Le Vétéran)** : C'est le standard de l'industrie depuis des années. Ultra-robuste et capable de s'adapter à toutes les situations, il est puissant mais demande une configuration minutieuse (gestion de multiples certificats et clés de sécurité).
- **WireGuard (Le Challenger)** : C'est la nouvelle révolution des réseaux. Pensé pour être minimaliste, furtif et incroyablement rapide, il se configure en quelques lignes seulement, balayant la complexité des anciens systèmes.

L'objectif de cette étude est double : déployer ces deux solutions de bout en bout pour connecter des réseaux distants, et observer par la pratique le "choc des générations". Nous verrons comment le monde des réseaux évolue d'une architecture lourde et complexe (OpenVPN) vers une simplicité absolue et redoutablement efficace (WireGuard).

Architecture



Dans ce TP les machines utilisées sont des machines virtuelles Debian 12 sous VMWare.

Partie 1 : Préparation de l'architecture et Routage

1.1 Configuration des machines

C1

```
root@debian:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:e3:fb:94 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
3: ens34: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:e3:fb:9e brd ff:ff:ff:ff:ff:ff
    altname enp2s2
    inet 192.168.10.1/24 brd 192.168.10.255 scope global ens34
        valid_lft forever preferred_lft forever
root@debian:~# ip r
default via 192.168.10.3 dev ens34 onlink
192.168.10.0/24 dev ens34 proto kernel scope link src 192.168.10.1
root@debian:~#
```

S3

```
root@debian:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:99:00:3a brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.80.234/24 brd 192.168.80.255 scope global dynamic noprefixroute ens33
        valid_lft 1465sec preferred_lft 1465sec
    inet6 fe80::3092:8d47:418a:8572/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: ens34: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:99:00:44 brd ff:ff:ff:ff:ff:ff
    altname enp2s2
    inet 192.168.10.3/24 brd 192.168.10.255 scope global ens34
        valid_lft forever preferred_lft forever
root@debian:~# ip r
default via 192.168.80.2 dev ens33 proto dhcp src 192.168.80.234 metric 100
192.168.10.0/24 dev ens34 proto kernel scope link src 192.168.10.3
192.168.80.0/24 dev ens33 proto kernel scope link src 192.168.80.234 metric 100
```

Activation du routage : ***sudo sysctl -w net.ipv4.ip_forward=1***

S4

```
root@debian:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:4d:16:68 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.80.236/24 brd 192.168.80.255 scope global dynamic noprefixroute ens33
        valid_lft 1465sec preferred_lft 1465sec
    inet6 fe80::1865:bf7f:7074:9894/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: ens34: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:4d:16:72 brd ff:ff:ff:ff:ff:ff
    altname enp2s2
    inet 192.168.30.4/24 brd 192.168.30.255 scope global ens34
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe4d:1672/64 scope link
        valid_lft forever preferred_lft forever
root@debian:~# ip r
default via 192.168.80.2 dev ens33 proto dhcp src 192.168.80.236 metric 100
192.168.10.0/24 via 192.168.80.234 dev ens33
192.168.30.0/24 dev ens34 proto kernel scope link src 192.168.30.4
192.168.80.0/24 dev ens33 proto kernel scope link src 192.168.80.236 metric 100
```

Activation du routage: ***sudo sysctl -w net.ipv4.ip_forward=1***

S5

```
root@debian:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:52:93:c4 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
3: ens34: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:52:93:ce brd ff:ff:ff:ff:ff:ff
    altname enp2s2
    inet 192.168.30.5/24 brd 192.168.30.255 scope global ens34
        valid_lft forever preferred_lft forever
root@debian:~# ip r
default via 192.168.30.4 dev ens34 onlink
192.168.30.0/24 dev ens34 proto kernel scope link src 192.168.30.5
```

1.2 Explication du routage:

- S3 (192.168.10.3) : Aucune route statique n'a été ajoutée. Sa route par défaut pointe vers la passerelle NAT de VMware pour avoir accès à Internet.
- S4 (192.168.30.4) : Sa route par défaut pointe également vers la passerelle NAT de VMware pour accéder à Internet. Et une route statique vers le réseau C1. Cela permettra d'indiquer à S4 que pour joindre le réseau de C1 (192.168.10.0/24), il doit passer par l'interface de S3 qui est connectée au réseau R-NAT, validant ainsi le ping S4/R-NAT depuis C1.
- C1 (192.168.10.1): Sa route par défaut est S3 (192.168.10.3). Cela permet de valider le ping S4 depuis C1 sans avoir à créer de route statique sur S3 (S3 forwardera naturellement les requêtes de C1 vers le réseau R-NAT).
- S5 (192.168.30.5) : S5 est un serveur interne. Sa passerelle par défaut est S4 (192.168.30.4). Le ping de C1 vers S5 échoue car le réseau R-NAT (entre S3 et S4) ne connaît pas la route de retour vers le réseau R1 (192.168.10.0), et les paquets sont dropped.
- Activation du routage des paquets sur S3 et S4

1.3 Test des pings

- ***ping S3/R1 depuis C1 est OK***

```
root@debian:~# ping -c 4 192.168.10.3
PING 192.168.10.3 (192.168.10.3) 56(84) bytes of data.
64 bytes from 192.168.10.3: icmp_seq=1 ttl=64 time=1.45 ms
64 bytes from 192.168.10.3: icmp_seq=2 ttl=64 time=1.30 ms
64 bytes from 192.168.10.3: icmp_seq=3 ttl=64 time=1.23 ms
64 bytes from 192.168.10.3: icmp_seq=4 ttl=64 time=0.905 ms

--- 192.168.10.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 0.905/1.220/1.450/0.199 ms
```

- ***ping S4/R-NAT depuis S3 est OK***

```
root@debian:~# ping -c 4 192.168.80.236
PING 192.168.80.236 (192.168.80.236) 56(84) bytes of data.
64 bytes from 192.168.80.236: icmp_seq=1 ttl=64 time=0.814 ms
64 bytes from 192.168.80.236: icmp_seq=2 ttl=64 time=0.801 ms
64 bytes from 192.168.80.236: icmp_seq=3 ttl=64 time=1.23 ms
64 bytes from 192.168.80.236: icmp_seq=4 ttl=64 time=0.874 ms

--- 192.168.80.236 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3044ms
rtt min/avg/max/mdev = 0.801/0.929/1.227/0.174 ms
```

- ***ping S4/R-NAT depuis C1 est OK***

```
root@debian:~# ping -c 4 192.168.80.236
PING 192.168.80.236 (192.168.80.236) 56(84) bytes of data.
64 bytes from 192.168.80.236: icmp_seq=1 ttl=63 time=2.49 ms
64 bytes from 192.168.80.236: icmp_seq=2 ttl=63 time=1.79 ms
64 bytes from 192.168.80.236: icmp_seq=3 ttl=63 time=1.74 ms
64 bytes from 192.168.80.236: icmp_seq=4 ttl=63 time=1.88 ms

--- 192.168.80.236 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 1.742/1.974/2.487/0.300 ms
```

- ***ping 194.199.90.1 depuis S3 et S4 sont OK***

```
root@debian:~# ping -c 4 194.199.90.1
PING 194.199.90.1 (194.199.90.1) 56(84) bytes of data.
64 bytes from 194.199.90.1: icmp_seq=1 ttl=128 time=15.7 ms
64 bytes from 194.199.90.1: icmp_seq=2 ttl=128 time=8.17 ms
64 bytes from 194.199.90.1: icmp_seq=3 ttl=128 time=7.34 ms
64 bytes from 194.199.90.1: icmp_seq=4 ttl=128 time=11.3 ms

--- 194.199.90.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 7.344/10.631/15.709/3.282 ms
```

```
root@debian:~# ping -c 4 194.199.90.1
PING 194.199.90.1 (194.199.90.1) 56(84) bytes of data.
64 bytes from 194.199.90.1: icmp_seq=1 ttl=128 time=13.6 ms
64 bytes from 194.199.90.1: icmp_seq=2 ttl=128 time=13.4 ms
64 bytes from 194.199.90.1: icmp_seq=3 ttl=128 time=12.9 ms
64 bytes from 194.199.90.1: icmp_seq=4 ttl=128 time=10.1 ms

--- 194.199.90.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 10.080/12.475/13.601/1.409 ms
```


- ***ping S5 depuis S4 est OK***

```
root@debian:~# ping -c 4 192.168.30.5
PING 192.168.30.5 (192.168.30.5) 56(84) bytes of data.
64 bytes from 192.168.30.5: icmp_seq=1 ttl=64 time=3.40 ms
64 bytes from 192.168.30.5: icmp_seq=2 ttl=64 time=0.821 ms
64 bytes from 192.168.30.5: icmp_seq=3 ttl=64 time=2.04 ms
64 bytes from 192.168.30.5: icmp_seq=4 ttl=64 time=0.784 ms

--- 192.168.30.5 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 0.784/1.763/3.404/1.074 ms
```

- ***ping S5 depuis C1 n'est pas OK (c'est normal et voulu)***

```
root@debian:~# ping -c 4 192.168.30.5
PING 192.168.30.5 (192.168.30.5) 56(84) bytes of data.

--- 192.168.30.5 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3069ms
```

Partie 2 : Mise en place d'OpenVPN

Dans cette partie les fichiers de configuration PKI (*ca.crt*, *openVPN.crt*, *openVPN.key*, *dh.pem*, *ta.key*) d'OpenVPN ont été fournis donc on a pas besoin de générer de nouveau le certificat et le couple de clé privé et public:

Explication des fichiers PKI d'OpenVPN:

- **ca.crt (Certificat de l'Autorité de Certification) :**
 - **Le rôle :** C'est le "juge de confiance". Il est installé sur le serveur et tous les clients. Il permet à chacun de vérifier que l'interlocuteur en face utilise bien un certificat légitime, signé par l'entreprise, et non un faux.
- **openVPN.crt (Certificat Public du serveur/client) :**
 - **Le rôle :** C'est la "carte d'identité publique". Le serveur (ou le client) l'envoie à l'autre partie pour prouver son identité. Il contient une clé publique qui permet de chiffrer des données que seul le propriétaire de la carte pourra lire.
- **openVPN.key (Clé Privée) :**
 - **Le rôle :** C'est le "secret absolu". Ce fichier ne doit **jamais** quitter la machine à laquelle il appartient. Il permet à la machine de déchiffrer les messages qui lui sont adressés et de prouver de manière irréfutable que c'est bien elle qui a envoyé le certificat.
- **dh.pem (Paramètres Diffie-Hellman) :**
 - **Le rôle :** C'est le "générateur de code secret". Il est utilisé uniquement par le serveur. Il fournit la base mathématique qui permet au client et au serveur de se mettre d'accord sur une clé de session temporaire de manière totalement sécurisée, même si quelqu'un espionne le réseau au moment de leur connexion (Perfect Forward Secrecy).
- **ta.key (Clé TLS-Auth / HMAC)**
 - **Le rôle :** C'est le "videur à la porte". C'est une clé secrète identique partagée entre le client et le serveur (HMAC). Si le serveur reçoit une requête de connexion qui ne possède pas la signature mathématique de cette clé, il jette le paquet à la poubelle sans même essayer de le lire. Cela rend votre VPN invisible aux scanners de ports et le protège contre les attaques de type déni de service (DDoS).

2.1 Configuration VPN

- modification du fichier */etc/openvpn/server.conf* sur S4

```
GNU nano 7.2 /etc/openvpn/server/server.conf *
# Fichier server.conf sur S4
port 1194
proto udp
dev tun

# Chemins vers les certificats et clés fournis
ca ca.crt
cert OpenVPN.crt
key OpenVPN.key
dh dh.pem
tls-auth ta.key 0

# Paramètres du réseau VPN (le tunnel)
server 10.8.0.0 255.255.255.0

#log
keepalive 10 120
cipher AES-256-CBC
persist-key
persist-tun
status openvpn-status.log
verb 3
```

- On Démarre le service côté serveur :
 - *sudo systemctl start openvpn-server@server*

C1

```
GNU nano 7.2 /etc/openvpn/client/client.conf *
# Fichier client.conf sur C1
client
dev tun
proto udp

# L'IP de S4
remote 192.168.80.236 1194

resolv-retry infinite
nobind
persist-key
persist-tun

# Chemins vers les certificats et clés du client
ca ca.crt
cert client1.crt
key client1.key
tls-auth ta.key 1

cipher AES-256-CBC
verb 3
```

- Démarrez le client :
 - *sudo openvpn --config /etc/openvpn/client/client.conf*

- IP S4 sur le VPN : **10.8.0.1**
- IP C1 sur le VPN : **10.8.0.6**

```
4: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 500
    link/none
    inet 10.8.0.6 peer 10.8.0.5/32 scope global tun0
        valid_lft forever preferred_lft forever
    inet6 fe80::5e78:994a:489:2b01/64 scope link stable-privacy
        valid_lft forever preferred_lft forever
```

```
debian@debian:/etc/openvpn/client$ netstat -rn
Table de routage IP du noyau
Destination      Passerelle      Genmask          Indic   MSS  Fenêtre  irtt  Iface
0.0.0.0          192.168.10.3    0.0.0.0          UG      0 0      0  ens34
10.8.0.1         10.8.0.5        255.255.255.255 UGH     0 0      0  tun0
10.8.0.5         0.0.0.0         255.255.255.255 UH      0 0      0  tun0
192.168.10.0     0.0.0.0         255.255.255.0   U       0 0      0  ens34
```

```
debian@debian:/etc/openvpn/client$ ip r
default via 192.168.10.3 dev ens34 onlink
10.8.0.1 via 10.8.0.5 dev tun0
10.8.0.5 dev tun0 proto kernel scope link src 10.8.0.6
192.168.10.0/24 dev ens34 proto kernel scope link src 192.168.10.1
debian@debian:/etc/openvpn/client$
```

- **ping 192.168.30.5 (S5)**

```
debian@debian:/etc/openvpn/client$ ping -c 4 192.168.30.5
PING 192.168.30.5 (192.168.30.5) 56(84) bytes of data.

--- 192.168.30.5 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3076ms
```

Explication de l'échec du ping 192.168.30.5 depuis C1 : Actuellement, C1 ne connaît pas la route pour joindre 192.168.30.0/24 via le VPN

- **Correction pour faire passer le SSH et le Ping par le VPN:**
 - En ligne de commande: Sur C1, on ajoute une route static:

■ **ip route add 192.168.30.0/24 via 10.8.0.5**

```
root@debian:/etc/openvpn/client# systemctl restart networking
root@debian:/etc/openvpn/client# ping -c 4 192.168.30.5
PING 192.168.30.5 (192.168.30.5) 56(84) bytes of data.
64 bytes from 192.168.30.5: icmp_seq=1 ttl=63 time=5.02 ms
64 bytes from 192.168.30.5: icmp_seq=2 ttl=63 time=3.97 ms
64 bytes from 192.168.30.5: icmp_seq=3 ttl=63 time=3.82 ms
64 bytes from 192.168.30.5: icmp_seq=4 ttl=63 time=3.77 ms

--- 192.168.30.5 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 3.772/4.144/5.019/0.509 ms
```

- Validant ainsi le ping et le ssh de C1 vers le S5 passant par le tunnel

Interface tun0 sur C1 => requet ICMP en clair sur le l'interface virtuel d'OpenVPN

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.8.0.6	192.168.30.5	ICMP	84	Echo (ping) request
2	0.005908075	192.168.30.5	10.8.0.6	ICMP	84	Echo (ping) reply
3	1.001969691	10.8.0.6	192.168.30.5	ICMP	84	Echo (ping) request
4	1.006177431	192.168.30.5	10.8.0.6	ICMP	84	Echo (ping) reply
5	2.004441879	10.8.0.6	192.168.30.5	ICMP	84	Echo (ping) request
6	2.008868233	192.168.30.5	10.8.0.6	ICMP	84	Echo (ping) reply
7	3.007618572	10.8.0.6	192.168.30.5	ICMP	84	Echo (ping) request
8	3.013678989	192.168.30.5	10.8.0.6	ICMP	84	Echo (ping) reply

Même la connexion ssh passe par le tunnel.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.8.0.6	192.168.30.5	TCP	60	37274 → 22 [SYN] Seq=0 Win=64240 L
2	0.004194399	192.168.30.5	10.8.0.6	TCP	60	22 → 37274 [SYN, ACK] Seq=0 Ack=1
3	0.004238974	10.8.0.6	192.168.30.5	TCP	52	37274 → 22 [ACK] Seq=1 Ack=1 Win=6
4	0.005037890	10.8.0.6	192.168.30.5	SSHv2	84	Client: Protocol (SSH-2.0-OpenSSH
5	0.009667305	192.168.30.5	10.8.0.6	TCP	52	22 → 37274 [ACK] Seq=1 Ack=33 Win=
6	0.040693567	192.168.30.5	10.8.0.6	SSHv2	84	Server: Protocol (SSH-2.0-OpenSSH
7	0.040989929	10.8.0.6	192.168.30.5	TCP	52	37274 → 22 [ACK] Seq=33 Ack=33 Win=
8	0.041908408	10.8.0.6	192.168.30.5	TCP	1440	37274 → 22 [ACK] Seq=33 Ack=33 Win=
9	0.041980265	10.8.0.6	192.168.30.5	SSHv2	168	Client: Key Exchange Init
10	0.046325630	192.168.30.5	10.8.0.6	SSHv2	1132	Server: Key Exchange Init
11	0.047099950	192.168.30.5	10.8.0.6	TCP	52	22 → 37274 [ACK] Seq=1113 Ack=1537
12	0.086909830	10.8.0.6	192.168.30.5	TCP	52	37274 → 22 [ACK] Seq=1537 Ack=1113
13	0.188302415	10.8.0.6	192.168.30.5	SSHv2	1260	Client: Diffie-Hellman Key Exchange

interface physique ens33 sur C1 => les paquets icmp sont encapsulés par le OpenVPN

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.80.238	192.168.10.1	OpenVPN	82	MessageType: P_DATA V2
2	0.000422178	192.168.10.1	192.168.80.238	OpenVPN	82	MessageType: P_DATA V2
15	10.105747210	192.168.80.238	192.168.10.1	OpenVPN	82	MessageType: P_DATA V2
16	10.106171271	192.168.10.1	192.168.80.238	OpenVPN	82	MessageType: P_DATA V2
17	10.311688087	192.168.10.1	192.168.80.238	OpenVPN	150	MessageType: P_DATA V2
20	10.316278284	192.168.80.238	192.168.10.1	OpenVPN	150	MessageType: P_DATA V2
23	11.313924604	192.168.10.1	192.168.80.238	OpenVPN	150	MessageType: P_DATA V2
26	11.317062130	192.168.80.238	192.168.10.1	OpenVPN	150	MessageType: P_DATA V2
27	12.315567828	192.168.10.1	192.168.80.238	OpenVPN	150	MessageType: P_DATA V2
30	12.320003257	192.168.80.238	192.168.10.1	OpenVPN	150	MessageType: P_DATA V2
34	13.317789391	192.168.10.1	192.168.80.238	OpenVPN	150	MessageType: P_DATA V2
37	13.321070386	192.168.80.238	192.168.10.1	OpenVPN	150	MessageType: P_DATA V2

Interface R3 sur S4 => S5 ne connaît pas l'ip de C1, connaît uniquement l'ip du VPN qu'il utilise

No.	Time	Source	Destination	Protocol	Length	Info
3	2.547126509	10.8.0.6	192.168.30.5	ICMP	98	Echo (ping) request
4	2.548636151	192.168.30.5	10.8.0.6	ICMP	98	Echo (ping) reply
7	3.548727983	10.8.0.6	192.168.30.5	ICMP	98	Echo (ping) request
8	3.549573650	192.168.30.5	10.8.0.6	ICMP	98	Echo (ping) reply
11	4.551250998	10.8.0.6	192.168.30.5	ICMP	98	Echo (ping) request
12	4.552235648	192.168.30.5	10.8.0.6	ICMP	98	Echo (ping) reply
16	5.554321359	10.8.0.6	192.168.30.5	ICMP	98	Echo (ping) request
17	5.557059300	192.168.30.5	10.8.0.6	ICMP	98	Echo (ping) reply

Pour éviter de configurer manuellement la route statique sur C1 à chaque nouvelle connexion, nous allons l'intégrer au serveur VPN. Ainsi, le serveur indique automatiquement au client que pour joindre S5, il doit utiliser la passerelle du tunnel (10.8.0.5).

- Modification du fichier **server.conf** (sur S4): on ajoute la ligne
 - **push "route 192.168.30.0 255.255.255.0"**
 - Cette commande permet de d'envoyer les informations du routage au client dès qu'il se connecte. Le serveur lui envoie cet ordre de routage. C1 ajoute alors lui-même la route dans sa table de routage sans qu'on ait à taper de commande ip route manuellement.

```
2026-04-22 11:42:48 net_addr_ptp_v4_add: 10.8.0.6 peer 10.8.0.5 dev tun0
2026-04-22 11:42:48 net_route_v4_add: 192.168.30.0/24 via 10.8.0.5 dev [NULL] table 0 metric -1
2026-04-22 11:42:48 net_route_v4_add: 10.8.0.1/32 via 10.8.0.5 dev [NULL] table 0 metric -1
```

- ```
root@debian:/etc/openvpn/client# ip r
default via 192.168.10.3 dev ens34 onlink
10.8.0.1 via 10.8.0.5 dev tun0
10.8.0.5 dev tun0 proto kernel scope link src 10.8.0.6
192.168.10.0/24 dev ens34 proto kernel scope link src 192.168.10.1
192.168.30.0/24 via 10.8.0.5 dev tun0
```

## Partie 3 : Étude du routage global

### 3.1 Configuration du VPN pour tous les paquets sortant du C1

- Tous les paquets sortant de C1 (y compris vers Internet) devront être encapsulés et envoyés vers S4. S4 devra agir comme routeur NAT pour ce trafic virtuel vers Internet afin de garantir le chemin de retour, sinon les serveurs web sur Internet ne sauront pas comment répondre à l'IP privée du VPN 10.8.0.1

```
root@debian:/etc/openvpn/client# ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
```

```
--- 8.8.8.8 ping statistics ---
```

```
4 packets transmitted, 0 received, 100% packet loss, time 3060ms
```

| No. | Time        | Source       | Destination | Protocol | Length | Info                |
|-----|-------------|--------------|-------------|----------|--------|---------------------|
| 3   | 3.337233206 | 192.168.10.1 | 8.8.8.8     | ICMP     | 98     | Echo (ping) request |
| 4   | 4.356468506 | 192.168.10.1 | 8.8.8.8     | ICMP     | 98     | Echo (ping) request |
| 8   | 5.380501560 | 192.168.10.1 | 8.8.8.8     | ICMP     | 98     | Echo (ping) request |
| 9   | 6.404310867 | 192.168.10.1 | 8.8.8.8     | ICMP     | 98     | Echo (ping) request |

On voit que le ping ne passe pas, sans traduction d'adresse ip au niveau du S4 et utilise l'interface physique de C1 sans passer par le tunnel VPN.

- Dans le fichier **server.conf** sur S4, on ajoute la directive:
  - push "redirect-gateway def1" (0.0.0.0/0)**: à pour but de forcer tout le trafic sortant de C1 (y compris sa navigation Web vers Internet) à entrer dans le tunnel VPN **tun0**
- Ajout du NAT (Masquerade) sur l'interface de S4 connectée au réseau R-NAT permet de remplacer l'adresse privée de C1 par l'adresse "publique" (ou du moins, celle connectée à la passerelle Internet) de S4:
  - dans le fichier **/etc/nftables.conf** on ajoute les lignes suivantes:

```
traduction d'adresse nat
table ip nat {
 chain postrouting {
 type nat hook postrouting priority 100; policy accept;

 # Règle NAT pour l'accès global à Internet (Tout le trafic VPN)
 ip saddr 10.8.0.0/24 oifname ens33 masquerade
 }
}
```

| No. | Time        | Source   | Destination | Protocol | Length | Info                |
|-----|-------------|----------|-------------|----------|--------|---------------------|
| 1   | 0.000000000 | 10.8.0.6 | 8.8.8.8     | ICMP     | 84     | Echo (ping) request |
| 2   | 0.018186022 | 8.8.8.8  | 10.8.0.6    | ICMP     | 84     | Echo (ping) reply   |
| 3   | 1.002127406 | 10.8.0.6 | 8.8.8.8     | ICMP     | 84     | Echo (ping) request |
| 4   | 1.040265419 | 8.8.8.8  | 10.8.0.6    | ICMP     | 84     | Echo (ping) reply   |
| 5   | 2.004091020 | 10.8.0.6 | 8.8.8.8     | ICMP     | 84     | Echo (ping) request |
| 6   | 2.014717346 | 8.8.8.8  | 10.8.0.6    | ICMP     | 84     | Echo (ping) reply   |
| 7   | 3.005635663 | 10.8.0.6 | 8.8.8.8     | ICMP     | 84     | Echo (ping) request |
| 8   | 3.019020153 | 8.8.8.8  | 10.8.0.6    | ICMP     | 84     | Echo (ping) reply   |

- On voit ainsi que le ping vers google passe bien par le server VPN (10.8.0.6)

## 3.2 Explication du Table de routage C1

```
root@debian:/etc/openvpn/client# netstat -rn
Table de routage IP du noyau
Destination Passerelle Genmask Indic MSS Fenêtre irtt Iface
0.0.0.0 10.8.0.5 128.0.0.0 UG 0 0 0 tun0
0.0.0.0 192.168.10.3 0.0.0.0 UG 0 0 0 ens34
10.8.0.1 10.8.0.5 255.255.255.255 UGH 0 0 0 tun0
10.8.0.5 0.0.0.0 255.255.255.255 UH 0 0 0 tun0
128.0.0.0 10.8.0.5 128.0.0.0 UG 0 0 0 tun0
192.168.10.0 0.0.0.0 255.255.255.0 U 0 0 0 ens34
192.168.30.0 10.8.0.5 255.255.255.0 UG 0 0 0 tun0
192.168.80.238 192.168.10.3 255.255.255.255 UGH 0 0 0 ens34
root@debian:/etc/openvpn/client#
```

### 1. Le détournement du trafic global (Les deux routes /1)

- 0.0.0.0 (Genmask 128.0.0.0) vers passerelle 10.8.0.5 (Interface tun0)
- 128.0.0.0 (Genmask 128.0.0.0) vers passerelle 10.8.0.5 (Interface tun0)
  - **Explication :** C'est le cœur de l'astuce def1. Au lieu d'effacer la route par défaut existante, OpenVPN crée deux nouvelles routes très larges qui englobent l'intégralité des adresses IPv4 existantes (la première couvre les adresses de 0 à 127, et la seconde de 128 à 255). Comme leur masque réseau (128.0.0.0, soit un sous-réseau /1) est plus précis que la route par défaut standard (0.0.0.0 en /0), elles deviennent prioritaires. Tout le trafic vers Internet est donc "aspiré" vers le tunnel VPN (tun0).

### 2. L'exception vitale (La route d'hôte vers le serveur VPN)

- 192.168.80.238 (Genmask 255.255.255.255) vers passerelle 192.168.10.3 (Interface ens34)
  - **Explication :** C'est une route d'hôte (le masque .255 indique une machine unique). L'IP 192.168.80.238 correspond à l'adresse "publique" de ton serveur VPN (S4 sur le réseau R-NAT). OpenVPN ajoute automatiquement cette route pour obliger les paquets *chiffrés* du tunnel à utiliser l'ancienne passerelle physique (S3). Sans cette exception, le tunnel VPN essaierait de s'envoyer lui-même dans le VPN, créant une boucle de routage infinie et coupant la connexion !

### 3. L'ancienne route par défaut (La route de secours)

- 0.0.0.0 (Genmask 0.0.0.0) vers passerelle 192.168.10.3 (Interface ens34)
  - **Explication :** C'est la route par défaut d'origine de ta machine C1, qui pointe vers le routeur S3. Elle n'a pas été supprimée, mais elle est ignorée par le système à cause des deux routes plus précises du VPN vues en point



1. Si on coupe le VPN, les deux routes en /1 disparaissent, et cette route redevient active instantanément.

#### 4. Les routes du réseau d'entreprise (Chemin Aller)

- 192.168.30.0 (Genmask 255.255.255.0) vers passerelle 10.8.0.5 (Interface tun0)
  - **Explication** : C'est la route qui a été poussée précédemment par le serveur pour permettre d'atteindre le réseau interne R3 (où se trouve le serveur S5).

#### 5. Les routes du réseau local physique

- 192.168.10.0 (Genmask 255.255.255.0) vers 0.0.0.0 (Interface ens34)
  - **Explication** : C'est la route qui indique que C1 est physiquement connecté au réseau R1. Le trafic destiné à d'autres machines sur ce même câble (comme 192.168.10.3) ne passe pas par le routeur, d'où la passerelle 0.0.0.0.

#### 6. Les routes de la topologie VPN (net30)

- 10.8.0.5 (Genmask 255.255.255.255) vers 0.0.0.0 (Interface tun0)
- 10.8.0.1 (Genmask 255.255.255.255) vers passerelle 10.8.0.5 (Interface tun0)
  - **Explication** : Ces routes sont liées à l'architecture net30 d'OpenVPN. Elles indiquent comment joindre l'adresse du serveur VPN 10.8.0.1 en passant par le "peer" (le bout du tuyau) virtuel qui t'a été assigné, 10.8.0.5

## Partie 4 : L'infrastructure DNS

### 4.1 Étape 1 : S3 en serveur DNS Cache (Bind9)

L'objectif ici est que S3 interroge Internet pour résoudre les noms publics (comme `www.univ-evry.fr`) et garde les réponses en mémoire.

Sur S3 :

1. Installation du serveur DNS Bind9 : **`sudo apt install bind9 bind9utils bind9-doc`**
2. Configuration des options globales pour autoriser les requêtes de C1 et faire office de cache. Édite `/etc/bind/named.conf.options` :

```
GNU nano 7.2 /etc/bind/named.conf.options
acl "trusted"{
 127.0.0.0/8;
 192.168.10.0/24; //reseau C1
};

options {
 directory "/var/cache/bind";
 recursion yes;
 allow-query {trusted;};
 forwarders{
 8.8.8.8; //dns de google
 };
 dnssec-validation auto;
 listen-on-v6 {any;};
}
```

3. Redémarre le service : **`sudo systemctl restart bind9.`**
4. Sur C1 (Test) : Modifie `/etc/resolv.conf` pour qu'il utilise S3 :
  - a. **`nameserver 192.168.10.3.`**
  - b. Test de la résolution : **`ping www.univ-evry.fr.`**

```
root@debian:/etc/openvpn/client# ping -c 4 www.univ-evry.fr
PING typo3.univ-evry.fr (194.199.84.39) 56(84) bytes of data.
64 bytes from typo3.univ-evry.fr (194.199.84.39): icmp_seq=1 ttl=127 time=11.8 ms
64 bytes from typo3.univ-evry.fr (194.199.84.39): icmp_seq=2 ttl=127 time=11.6 ms
64 bytes from typo3.univ-evry.fr (194.199.84.39): icmp_seq=3 ttl=127 time=11.5 ms
64 bytes from typo3.univ-evry.fr (194.199.84.39): icmp_seq=4 ttl=127 time=11.0 ms

--- typo3.univ-evry.fr ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 11.028/11.496/11.832/0.293 ms
```

- c. Récupération de l'IP: **`dig www.univ-evry.fr`**

```
root@debian:/etc/openvpn/client# dig www.univ-evry.fr

; <<>> DiG 9.18.16-1-deb12u1-Debian <<>> www.univ-evry.fr
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 24691
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 1232
;; COOKIE: c3c54927caaf3ea40100000069e8e7a4169dcbc20200a634 (good)
;; QUESTION SECTION:
;www.univ-evry.fr. IN A

;; ANSWER SECTION:
www.univ-evry.fr. 3012 IN CNAME typo3.univ-evry.fr.
typo3.univ-evry.fr. 3012 IN A 194.199.84.39

;; Query time: 4 msec
;; SERVER: 192.168.10.3#53(192.168.10.3) (UDP)
;; WHEN: Wed Apr 22 17:22:12 CEST 2026
;; MSG SIZE rcvd: 109
```

## 4.2 Étape 2 : Le Serveur DNS Interne (sur S5)

Pour que S4 puisse répondre aux requêtes du S5, sans ajouter une route statique sur S3. Nous sommes obligés d'ajouter une règle nat sur S4 afin de masquer l'ip de S5:

**ip saddr 192.168.30.0/24 oifname ens33 masquerade:** Cette règle permet à S4 remplace l'IP source 192.168.30.5 par sa propre IP 192.168.80.238.

S3 reçoit une requête venant de 192.168.80.238 (qu'il sache comment joindre puisqu'ils sont sur le même réseau R-NAT) et pourra répondre.

Sur S5 :

1. Installation Bind9 : **sudo apt install bind9.**
2. On configure les options (**/etc/bind/named.conf.options**) pour qu'il utilise S3 comme redirecteur (cela permet aux machines internes d'aller sur Internet si besoin

```
options {
 directory "/var/cache/bind";
 recursion yes;
 allow-query { any ; };
 forwarders{
 192.168.10.3; // IP de S3
 };
}
```

3. Déclare la zone interne dans **/etc/bind/named.conf.local**

```
GNU nano 7.2 /etc/bind/named.conf.local *
//
// Do any local configuration here
zone "interne.tp.org"{
 type master;
 file "/etc/bind/db.interne.tp.org";
};
```

4. Créons le fichier de zone **/etc/bind/db.interne.tp.org** (C'est ici qu'on associe les

```
GNU nano 7.2 /etc/bind/db.interne.tp.org
$TTL 604800
@ IN SOA ns1.interne.tp.org. admin.interne.tp.org. (
 2 ; Serial
 604800 ; Refresh
 86400 ; Retry
 2419200 ; Expire
 604800) ; Negative Cache TTL
;
@ IN NS ns1.interne.tp.org.
ns1 IN A 192.168.30.5
s5 IN A 192.168.30.5
s4 IN A 192.168.30.4
```

noms aux IPs ) :

```
root@debian:/etc/bind# host s4.interne.tp.org 127.0.0.1
Using domain server:
Name: 127.0.0.1
Address: 127.0.0.1#53
Aliases:
s4.interne.tp.org has address 192.168.30.4
root@debian:/etc/bind#
```

## 4.3 Étape 3 : Test de ssh de C1

- **Test depuis C1 : `ssh debian@s5.interne.tp.org`**

- Arrivez-vous à vous connecter ? Non !

```
root@debian:/etc/openvpn/client# ssh debian@s5.interne.tp.org
ssh: connect to host s5.interne.tp.org port 22: Connection refused
```

- **Pourquoi ?**

- C1 est configuré pour interroger S3 (le DNS cache) pour la résolution de noms. Cependant, S3 ne gère pas la zone interne.tp.org (elle est sur S5 ). Et S3 ne sait pas qu'il doit interroger S5 pour cette zone précise. S3 va donc chercher interne.tp.org sur Internet, ne trouvera rien, et dira à C1 : "Ce nom n'existe pas". La connexion SSH échoue avant même que le moindre paquet ne soit envoyé vers S5

- **Comment corriger ce problème ? (OpenVPN)**

- Il faut que, lorsque C1 se connecte au VPN, le serveur VPN (S4) dise à C1 : *"À partir de maintenant, n'utilise plus S3 comme serveur DNS, utilise le serveur DNS interne (S5) qui est accessible à travers le tunnel !"*.
- La correction se fait donc en poussant l'adresse du serveur DNS interne via le tunnel VPN.

- **La configuration (Sur S4) :**

- On ouvre le fichier `/etc/openvpn/server/server.conf` sur S4 et on ajoute la directive DHCP : **`push "dhcp-option DNS 192.168.30.5"`**
- On redémarre le serveur OpenVPN sur S4: **`sudo systemctl start openvpn-server@server`**
- Puis on Reconnecte le client C1: **`sudo openvpn --config /etc/openvpn/client/client.conf`**
- Puis on refait le **`ssh debian@s5.interne.tp.org`**

```
root@debian:/etc/openvpn/client# ssh debian@s5.interne.tp.org
debian@s5.interne.tp.org's password:
Linux debian 6.1.0-11-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.38-4 (2023-08-08) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Apr 22 21:38:26 2026 from 10.8.0.6
debian@debian:~$
```

```
GNU nano 7.2 /etc/resolv.conf
Dynamic resolv.conf(5) file for glibc resolver
DO NOT EDIT THIS FILE BY HAND -- YOUR CHANGES WILL BE OVERWRITTEN
127.0.0.53 is the systemd-resolved stub resolver
run "resolvectl status" to see details about the current configuration

nameserver 192.168.30.5
nameserver 192.168.80.2
search localdomain
```

## Configuration finale du serveur VPN (S4)

```
GNU nano 7.2 /etc/openvpn/server/server.conf
Fichier server.conf sur S4
port 1194
proto udp
dev tun

Chemins vers les certificats et clés fournis
ca ca.crt
cert OpenVPN.crt
key OpenVPN.key
dh dh.pem
tls-auth ta.key 0

Paramètres du réseau VPN (le tunnel)
server 10.8.0.0 255.255.255.0
push "route 192.168.30.0 255.255.255.0"
push "redirect-gateway def1"
push "dhcp-option DNS 192.168.30.5"
#log
keepalive 10 120
cipher AES-256-CBC
persist-key
persist-tun
status openvpn-status.log
verb 3
```

# Test de résilience (Coupure de l'interface physique) :

- Lancement d'un ping continu depuis C1 vers le réseau distant.
- Désactivation manuelle de l'interface physique du client (*sudo ip link set ens33 down*).
- Réactivation de l'interface (*sudo ip link set ens33 up*).
- **Observation** : L'objectif de cette manipulation est d'observer si le service OpenVPN est capable de restaurer la connexion de manière autonome, sans nécessiter un redémarrage manuel du client.

| No. | Time          | Source                 | Destination    | Protocol | Length | Info                                                                    |
|-----|---------------|------------------------|----------------|----------|--------|-------------------------------------------------------------------------|
| 38  | 19.5264658084 | 10.8.0.6               | 192.168.30.5   | ICMP     | 84     | Echo (ping) request id=0xfbcc, seq=19/4864, ttl=64 (reply in 39)        |
| 39  | 19.5310089994 | 192.168.30.5           | 10.8.0.6       | ICMP     | 84     | Echo (ping) reply id=0xfbcc, seq=19/4864, ttl=63 (request in 38)        |
| 40  | 20.528184928  | 10.8.0.6               | 192.168.30.5   | ICMP     | 84     | Echo (ping) request id=0xfbcc, seq=20/5120, ttl=64 (reply in 41)        |
| 41  | 20.533697220  | 192.168.30.5           | 10.8.0.6       | ICMP     | 84     | Echo (ping) reply id=0xfbcc, seq=20/5120, ttl=63 (request in 40)        |
| 42  | 21.530088774  | 10.8.0.6               | 192.168.30.5   | ICMP     | 84     | Echo (ping) request id=0xfbcc, seq=21/5376, ttl=64 (reply in 43)        |
| 43  | 21.534698477  | 192.168.30.5           | 10.8.0.6       | ICMP     | 84     | Echo (ping) reply id=0xfbcc, seq=21/5376, ttl=63 (request in 42)        |
| 44  | 22.530846826  | 10.8.0.6               | 192.168.30.5   | ICMP     | 84     | Echo (ping) request id=0xfbcc, seq=22/5632, ttl=64 (reply in 45)        |
| 45  | 22.535837688  | 192.168.30.5           | 10.8.0.6       | ICMP     | 84     | Echo (ping) reply id=0xfbcc, seq=22/5632, ttl=63 (request in 44)        |
| 46  | 23.532896933  | 10.8.0.6               | 192.168.30.5   | ICMP     | 84     | Echo (ping) request id=0xfbcc, seq=23/5888, ttl=64 (reply in 47)        |
| 47  | 23.537017540  | 192.168.30.5           | 10.8.0.6       | ICMP     | 84     | Echo (ping) reply id=0xfbcc, seq=23/5888, ttl=63 (request in 46)        |
| 48  | 24.535787837  | 10.8.0.6               | 192.168.30.5   | ICMP     | 84     | Echo (ping) request id=0xfbcc, seq=24/6144, ttl=64 (no response found!) |
| 49  | 24.535907251  | 10.8.0.6               | 192.168.80.238 | OpenVPN  | 136    | MessageType: P_DATA V2                                                  |
| 50  | 25.572345612  | 10.8.0.6               | 192.168.30.5   | ICMP     | 84     | Echo (ping) request id=0xfbcc, seq=25/6400, ttl=64 (no response found!) |
| 51  | 25.572718446  | 10.8.0.6               | 192.168.80.238 | OpenVPN  | 136    | MessageType: P_DATA V2                                                  |
| 52  | 26.592260993  | 10.8.0.6               | 192.168.30.5   | ICMP     | 84     | Echo (ping) request id=0xfbcc, seq=26/6656, ttl=64 (no response found!) |
| 53  | 26.592631162  | 10.8.0.6               | 192.168.80.238 | OpenVPN  | 136    | MessageType: P_DATA V2                                                  |
| 54  | 27.616599709  | 10.8.0.6               | 192.168.30.5   | ICMP     | 84     | Echo (ping) request id=0xfbcc, seq=27/6912, ttl=64 (no response found!) |
| 55  | 27.616965599  | 10.8.0.6               | 192.168.80.238 | OpenVPN  | 136    | MessageType: P_DATA V2                                                  |
| 56  | 28.639819885  | 10.8.0.6               | 192.168.30.5   | ICMP     | 84     | Echo (ping) request id=0xfbcc, seq=28/7168, ttl=64 (no response found!) |
| 57  | 28.640101049  | 10.8.0.6               | 192.168.80.238 | OpenVPN  | 136    | MessageType: P_DATA V2                                                  |
| 58  | 29.664428233  | 10.8.0.6               | 192.168.30.5   | ICMP     | 84     | Echo (ping) request id=0xfbcc, seq=29/7424, ttl=64 (no response found!) |
| 59  | 29.664693267  | 10.8.0.6               | 192.168.80.238 | OpenVPN  | 136    | MessageType: P_DATA V2                                                  |
| 60  | 30.688207011  | 10.8.0.6               | 192.168.30.5   | ICMP     | 84     | Echo (ping) request id=0xfbcc, seq=30/7680, ttl=64 (no response found!) |
| 61  | 30.688614240  | 10.8.0.6               | 192.168.80.238 | OpenVPN  | 136    | MessageType: P_DATA V2                                                  |
| 62  | 31.712363080  | 10.8.0.6               | 192.168.30.5   | ICMP     | 84     | Echo (ping) request id=0xfbcc, seq=31/7936, ttl=64 (no response found!) |
| 63  | 31.712863694  | 10.8.0.6               | 192.168.80.238 | OpenVPN  | 136    | MessageType: P_DATA V2                                                  |
| 64  | 32.736022939  | 10.8.0.6               | 192.168.30.5   | ICMP     | 84     | Echo (ping) request id=0xfbcc, seq=32/8192, ttl=64 (no response found!) |
| 65  | 32.736564880  | 10.8.0.6               | 192.168.80.238 | OpenVPN  | 136    | MessageType: P_DATA V2                                                  |
| 66  | 32.768148361  | fe80::2c3a:c884:138... | ff02::2        | ICMPv6   | 48     | Router Solicitation                                                     |
| 67  | 32.768680280  | 10.8.0.6               | 192.168.80.238 | OpenVPN  | 100    | MessageType: P_DATA V2                                                  |
| 68  | 33.760927736  | 10.8.0.6               | 192.168.30.5   | ICMP     | 84     | Echo (ping) request id=0xfbcc, seq=33/8448, ttl=64 (no response found!) |
| 69  | 33.761137578  | 10.8.0.6               | 192.168.80.238 | OpenVPN  | 136    | MessageType: P_DATA V2                                                  |
| 70  | 34.784073661  | 10.8.0.6               | 192.168.30.5   | ICMP     | 84     | Echo (ping) request id=0xfbcc, seq=34/8704, ttl=64 (no response found!) |
| 71  | 34.784465881  | 10.8.0.6               | 192.168.80.238 | OpenVPN  | 136    | MessageType: P_DATA V2                                                  |
| 72  | 35.807874362  | 10.8.0.6               | 192.168.30.5   | ICMP     | 84     | Echo (ping) request id=0xfbcc, seq=35/8960, ttl=64 (no response found!) |
| 73  | 35.808222010  | 10.8.0.6               | 192.168.80.238 | OpenVPN  | 136    | MessageType: P_DATA V2                                                  |

```

64 bytes from 192.168.30.5: icmp_seq=14 ttl=63 time=4.35 ms
64 bytes from 192.168.30.5: icmp_seq=15 ttl=63 time=5.27 ms
64 bytes from 192.168.30.5: icmp_seq=16 ttl=63 time=6.02 ms
64 bytes from 192.168.30.5: icmp_seq=17 ttl=63 time=4.84 ms
64 bytes from 192.168.30.5: icmp_seq=18 ttl=63 time=5.04 ms
64 bytes from 192.168.30.5: icmp_seq=19 ttl=63 time=4.58 ms
64 bytes from 192.168.30.5: icmp_seq=20 ttl=63 time=5.55 ms
64 bytes from 192.168.30.5: icmp_seq=21 ttl=63 time=4.65 ms
64 bytes from 192.168.30.5: icmp_seq=22 ttl=63 time=5.03 ms
64 bytes from 192.168.30.5: icmp_seq=23 ttl=63 time=4.19 ms
^C
--- 192.168.30.5 ping statistics ---
91 packets transmitted, 23 received, 74.7253% packet loss, time 91664ms
rtt min/avg/max/mdev = 3.594/4.877/6.910/0.714 ms

```

## Résultat et conclusion :

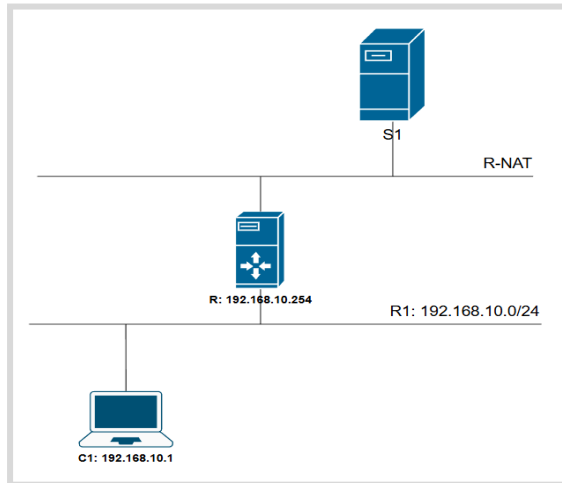
Contrairement aux attentes, le trafic ne reprend pas automatiquement après la réactivation de l'interface ens33. La connexion est définitivement interrompue et nécessite un redémarrage manuel du service OpenVPN sur le client.

**Explication technique :** Ce comportement s'explique par la nature "**stateful**" (avec état) d'OpenVPN et son intégration avec le système d'exploitation :

1. **Perte de routage :** Lors de la désactivation de l'interface physique (ens33), le noyau Linux supprime instantanément toutes les routes associées à cette carte. L'architecture de routage d'OpenVPN s'effondre.
2. **Rupture de la session TLS :** OpenVPN maintient une session cryptographique active avec le serveur. La coupure physique entraîne la perte des paquets de maintien en vie (keepalive). Le serveur et le client finissent par atteindre un délai d'attente (timeout) et considèrent la session TLS comme morte.
3. **Absence de reconnexion dynamique (sans options spécifiques) :** Lorsque l'interface ens33 revient, OpenVPN ne détecte pas dynamiquement ce changement d'état matériel pour relancer son processus de poignée de main (handshake) à partir de zéro. Le processus devient "zombie", attendant sur une interface virtuelle tun0 dont les routes sous-jacentes sont obsolètes.

# WireGuard

## Architecture



## Q1.

**Combien de fichiers ont été nécessaires ? Comparez avec la PKI d'OpenVPN**

- **Nombre de fichiers nécessaires avec WireGuard**
  - En exécutant la commande `wg genkey | tee privatekey | wg pubkey > publickey`, seulement **2 fichiers** ont été générés par machine :
    - Une clé privée (`privatekey`)
    - Une clé publique (`publickey`)

*(Soit un total de 4 fichiers pour établir un tunnel basique entre un client et un serveur).*

- **Comparaison avec la PKI d'OpenVPN**
  - **L'approche WireGuard (Simple et décentralisée)** : WireGuard s'inspire de SSH. Il n'y a pas d'Autorité de Certification centrale. Le client et le serveur s'identifient simplement en s'échangeant mutuellement leurs clés publiques (courbe elliptique Curve25519). C'est ce qu'on appelle du *Crypto-Routing*.
  - **L'approche OpenVPN (Lourde et centralisée)** : OpenVPN nécessite la mise en place d'une Infrastructure à Clés Publiques (PKI), souvent via l'outil `easy-rsa`. Pour que le tunnel fonctionne, on a dû manipuler au moins **7 fichiers différents**:
    - Le certificat de l'autorité (**`ca.crt`**) présent des deux côtés.
    - Les certificats publics signés (**`serveur.crt`, `client.crt`**).
    - Les clés privées associées (**`serveur.key`, `client.key`**).
    - Les paramètres Diffie-Hellman (**`dh.pem`**) pour l'échange sécurisé.
    - La clé de signature HMAC (**`ta.key`**) pour se protéger des scans.



## Configuration des machines:

### C1 (*ip a et ip route*)

```
2: ens34: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
 link/ether 00:0c:29:c6:9d:ae brd ff:ff:ff:ff:ff:ff
 altname enp2s2
 inet 192.168.10.1/24 brd 192.168.10.255 scope global ens34
 valid_lft forever preferred_lft forever
 inet6 fe80::20c:29ff:fec6:9dae/64 scope link
 valid_lft forever preferred_lft forever
3: wg0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1420 qdisc noqueue state UNKNOWN group default qlen 1000
 link/none
 inet 10.10.0.2/24 scope global wg0
 valid_lft forever preferred_lft forever
root@debian:~# ip r
default via 192.168.10.254 dev ens34 onlink
10.10.0.0/24 dev wg0 proto kernel scope link src 10.10.0.2
192.168.10.0/24 dev ens34 proto kernel scope link src 192.168.10.1
```

### C1 (config VPN */etc/wireguard/wg0.conf*)

```
GNU nano 7.2 /etc/wireguard/wg0.conf
[Interface]
Address = 10.10.0.2/24
PrivateKey = ODb1POSHGCPPXsKjzzU0nU65lSKZa1PiQ0dDjlw04UE=

[Peer]
C'est le serveur S4
PublicKey = MK6woNds/lzEzheVbuEInmLzTOBpbIQJVT94smpPj1w=
Endpoint = 192.168.80.245:51820
AllowedIPs = 10.10.0.0/24
```

### S1 (*ip a et ip route*)

```
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
 link/ether 00:0c:29:7d:aa:90 brd ff:ff:ff:ff:ff:ff
 altname enp2s1
 inet 192.168.80.245/24 brd 192.168.80.255 scope global dynamic ens33
 valid_lft 1359sec preferred_lft 1359sec
 inet6 fe80::20c:29ff:fe7d:aa90/64 scope link
 valid_lft forever preferred_lft forever
4: wg0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1420 qdisc noqueue state UNKNOWN group default qlen 1000
 link/none
 inet 10.10.0.1/24 scope global wg0
 valid_lft forever preferred_lft forever
root@debian:/etc/wireguard# ip r
default via 192.168.80.2 dev ens33
10.10.0.0/24 dev wg0 proto kernel scope link src 10.10.0.1
192.168.10.0/24 via 192.168.80.243 dev ens33
192.168.80.0/24 dev ens33 proto kernel scope link src 192.168.80.245
```

## S1 (config VPN */etc/wireguard/wg0.conf*)

```
GNU nano 7.2 /etc/wireguard/wg0.conf
[Interface]
Address = 10.10.0.1/24
ListenPort = 51820
PrivateKey = 4DcNS6ITYsXkWtPNxQDNpc+XdBS4l26xT2WrWOPiFHQ=

[Peer]
PublicKey = FwJAIYfqiC0pRy+S1K/Uv8hpsGjgmeGXXbzB/xh4Kng=
AllowedIPs = 10.10.0.2/32
```

Puis on démarre le tunnel des deux côtés: **sudo wg-quick up wg0**

## Test de ping VPN:

```
root@debian:~# wg show
interface: wg0
 public key: FwJAIYfqiC0pRy+S1K/Uv8hpsGjgmeGXXbzB/xh4Kng=
 private key: (hidden)
 listening port: 50009

peer: MK6woNds/lzEzheVbuEInmLzT0BpbIQJVT94smpPj1w=
 endpoint: 192.168.80.245:51820
 allowed ips: 10.10.0.0/24
 latest handshake: 7 seconds ago
 transfer: 1.18 KiB received, 4.21 KiB sent
root@debian:~# ping -c 4 10.10.0.1
PING 10.10.0.1 (10.10.0.1) 56(84) bytes of data.
64 bytes from 10.10.0.1: icmp_seq=1 ttl=64 time=2.31 ms
64 bytes from 10.10.0.1: icmp_seq=2 ttl=64 time=2.68 ms
64 bytes from 10.10.0.1: icmp_seq=3 ttl=64 time=2.68 ms
64 bytes from 10.10.0.1: icmp_seq=4 ttl=64 time=2.51 ms

--- 10.10.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 2.313/2.548/2.684/0.152 ms
```

| No. | Time        | Source    | Destination | Protocol | Length | Info                |
|-----|-------------|-----------|-------------|----------|--------|---------------------|
| 1   | 0.000000000 | 10.10.0.2 | 10.10.0.1   | ICMP     | 84     | Echo (ping) request |
| 2   | 0.002344096 | 10.10.0.1 | 10.10.0.2   | ICMP     | 84     | Echo (ping) reply   |
| 3   | 1.002637766 | 10.10.0.2 | 10.10.0.1   | ICMP     | 84     | Echo (ping) request |
| 4   | 1.005137661 | 10.10.0.1 | 10.10.0.2   | ICMP     | 84     | Echo (ping) reply   |
| 5   | 2.005128796 | 10.10.0.2 | 10.10.0.1   | ICMP     | 84     | Echo (ping) request |
| 6   | 2.007755513 | 10.10.0.1 | 10.10.0.2   | ICMP     | 84     | Echo (ping) reply   |
| 7   | 3.006407029 | 10.10.0.2 | 10.10.0.1   | ICMP     | 84     | Echo (ping) request |
| 8   | 3.008959134 | 10.10.0.1 | 10.10.0.2   | ICMP     | 84     | Echo (ping) reply   |

| udp.port==51820 |             |                |                |           |        |                                      |
|-----------------|-------------|----------------|----------------|-----------|--------|--------------------------------------|
| No.             | Time        | Source         | Destination    | Protocol  | Length | Info                                 |
| 6               | 4.277647721 | 192.168.10.1   | 192.168.80.245 | WireGu... | 170    | Transport Data, receiver=0x38E27E17, |
| 7               | 4.279779504 | 192.168.80.245 | 192.168.10.1   | WireGu... | 170    | Transport Data, receiver=0xDAA53A0C, |
| 10              | 5.280676101 | 192.168.10.1   | 192.168.80.245 | WireGu... | 170    | Transport Data, receiver=0x38E27E17, |
| 11              | 5.282552410 | 192.168.80.245 | 192.168.10.1   | WireGu... | 170    | Transport Data, receiver=0xDAA53A0C, |
| 13              | 6.282958364 | 192.168.10.1   | 192.168.80.245 | WireGu... | 170    | Transport Data, receiver=0x38E27E17, |
| 14              | 6.285085340 | 192.168.80.245 | 192.168.10.1   | WireGu... | 170    | Transport Data, receiver=0xDAA53A0C, |
| 15              | 7.284342018 | 192.168.10.1   | 192.168.80.245 | WireGu... | 170    | Transport Data, receiver=0x38E27E17, |
| 16              | 7.286462762 | 192.168.80.245 | 192.168.10.1   | WireGu... | 170    | Transport Data, receiver=0xDAA53A0C, |

**Observation du trafic et principe d'encapsulation :** L'écoute simultanée des interfaces wg0 et ens33 met en évidence le mécanisme d'encapsulation et de chiffrement de WireGuard.

- Sur l'interface virtuelle **wg0**, nous observons le trafic "utile" en clair (les requêtes et réponses ICMP entre les adresses IP du tunnel 10.10.0.2 et 10.10.0.1).
- Sur l'interface physique **ens33**, ce trafic ICMP n'est plus visible. Nous n'observons plus que des trames UDP (Port 51820) circulant entre les adresses IP physiques des machines. Le paquet ICMP d'origine a été chiffré et encapsulé par WireGuard pour former le "Transport Data". Cela prouve que le tunnel sécurise efficacement les données et masque la nature du trafic interne vis-à-vis du réseau physique.

## Q2.

*Modifiez AllowedIPs côté client à 0.0.0.0/0. Que se passe-t-il ? Quel est l'équivalent de la directive redirect-gateway d'OpenVPN ?*

**Redirection du trafic (AllowedIPs = 0.0.0.0/0)**

- **Que se passe-t-il ?** En modifiant AllowedIPs à 0.0.0.0/0 (qui représente l'intégralité de l'espace d'adressage IPv4), on indique à WireGuard d'acheminer **absolument tout le trafic sortant** de la machine client vers le tunnel VPN. L'outil **wg-quick** va automatiquement modifier la table de routage du client pour que la route par défaut pointe vers l'interface wg0.
- **L'équivalent OpenVPN :** C'est l'équivalent direct de la directive **redirect-gateway def1** que nous avons étudiée dans la partie 3 du TP.

Et bien sûr on oublie la règle nat, pour masquer l'ip privée du vpn

```
table ip nat {
 chain postrouting {
 type nat hook postrouting priority 100; policy accept;
 # Règle NAT pour l'accès global à Internet (Tout le trafic VPN)
 ip saddr 10.10.0.0/24 oifname ens33 masquerade
 }
}
```

## Ping 8.8.8.8 depuis C1: interface ens33 de C1

| No. | Time        | Source    | Destination | Protocol | Length | Info                |
|-----|-------------|-----------|-------------|----------|--------|---------------------|
| 1   | 0.000000000 | 10.10.0.2 | 8.8.8.8     | ICMP     | 84     | Echo (ping) request |
| 2   | 0.011148698 | 8.8.8.8   | 10.10.0.2   | ICMP     | 84     | Echo (ping) reply   |
| 3   | 1.003109611 | 10.10.0.2 | 8.8.8.8     | ICMP     | 84     | Echo (ping) request |
| 4   | 1.011898205 | 8.8.8.8   | 10.10.0.2   | ICMP     | 84     | Echo (ping) reply   |
| 5   | 2.005310955 | 10.10.0.2 | 8.8.8.8     | ICMP     | 84     | Echo (ping) request |
| 6   | 2.015738093 | 8.8.8.8   | 10.10.0.2   | ICMP     | 84     | Echo (ping) reply   |
| 7   | 3.007318218 | 10.10.0.2 | 8.8.8.8     | ICMP     | 84     | Echo (ping) request |
| 8   | 3.030876049 | 8.8.8.8   | 10.10.0.2   | ICMP     | 84     | Echo (ping) reply   |

On voit bien que le trafic vers internet passe par le Wireguard

### Table de routage C1

```
root@debian:/etc/wireguard# netstat -rn
Table de routage IP du noyau
Destination Passerelle Genmask Indic MSS Fenêtre irtt Iface
0.0.0.0 192.168.10.254 0.0.0.0 UG 0 0 0 ens34
10.10.0.0 0.0.0.0 255.255.255.0 U 0 0 0 wg0
192.168.10.0 0.0.0.0 255.255.255.0 U 0 0 0 ens34
```

## Q3.

*Désactivez l'interface physique du client (sudo ip link set ens33 down), puis réactivez-la. Le tunnel se rétablit-il automatiquement ? Comparez avec le comportement d'OpenVPN*

```
root@debian:/etc/wireguard# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=127 time=21.7 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=127 time=26.4 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=127 time=12.9 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=127 time=10.7 ms
64 bytes from 8.8.8.8: icmp_seq=25 ttl=127 time=11.4 ms
64 bytes from 8.8.8.8: icmp_seq=26 ttl=127 time=18.7 ms
64 bytes from 8.8.8.8: icmp_seq=27 ttl=127 time=11.2 ms
64 bytes from 8.8.8.8: icmp_seq=28 ttl=127 time=15.7 ms
64 bytes from 8.8.8.8: icmp_seq=29 ttl=127 time=11.3 ms
64 bytes from 8.8.8.8: icmp_seq=30 ttl=127 time=42.6 ms
64 bytes from 8.8.8.8: icmp_seq=31 ttl=127 time=16.7 ms
64 bytes from 8.8.8.8: icmp_seq=32 ttl=127 time=12.2 ms
64 bytes from 8.8.8.8: icmp_seq=33 ttl=127 time=21.3 ms
64 bytes from 8.8.8.8: icmp_seq=34 ttl=127 time=10.8 ms
64 bytes from 8.8.8.8: icmp_seq=35 ttl=127 time=9.43 ms
64 bytes from 8.8.8.8: icmp_seq=36 ttl=127 time=13.3 ms
^C
--- 8.8.8.8 ping statistics ---
36 packets transmitted, 16 received, 55.5556% packet loss, time 35512ms
rtt min/avg/max/mdev = 9.425/16.646/42.614/8.192 ms
```

### Observation:

- On lance un vers 8.8.8.8 en contitue puis on désactive l'interface ens33 (***sudo ip link set ens33 down***): on remarque le ping est interrompus pas icmp reply
- puis on réactive l'interface avec la commande ***systemctl restart networking***: on remarque que le ping reprend( icmp reply)

### Explication:

#### Rétablissement du tunnel (Stateless vs Stateful)

- **Le tunnel se rétablit-il automatiquement ? Oui, de manière instantanée et transparente.** Dès que l'interface physique ens33 remonte, les paquets recommencent à circuler comme si de rien n'était.
- **Comparaison avec OpenVPN:**
  - **WireGuard est "Stateless" (sans état) :** Il fonctionne exactement comme UDP (sur lequel il repose). Il n'y a pas de notion de "connexion" maintenue en permanence. Si l'interface physique coupe, WireGuard arrête juste d'envoyer des paquets. Quand elle revient, il reprend l'envoi. Le changement d'état réseau n'affecte pas le tunnel.
  - **OpenVPN est "Stateful" (avec état) :** Il repose sur une session sécurisée TLS (comme le HTTPS). Si l'interface physique est coupée, les paquets sont perdus, le serveur et le client détectent un "timeout", et la session TLS s'effondre. Lorsque l'interface revient, OpenVPN doit recommencer à zéro : renégocier les clés, vérifier les certificats, et refaire un "handshake" complet, ce qui prend plusieurs secondes et coupe les connexions actives de l'utilisateur.

## Q4.

*Depuis une machine non configurée comme peer, scannez le port 51820 du serveur avec nmap. Le serveur répond-il ? Expliquez le principe de stealth de WireGuard.*

```
root@debian:~# nmap -p 51820 192.168.80.245
Starting Nmap 7.93 (https://nmap.org) at 2026-04-23 21:05 CEST
Nmap scan report for 192.168.80.245
Host is up (0.00084s latency).

PORT STATE SERVICE
51820/tcp closed unknown
MAC Address: 00:0C:29:7D:AA:90 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.35 seconds
```

- **Le serveur répond-il au scan Nmap ? Non, il ne répond absolument rien.**  
Nmap affiche que le port 51820 est fermé
- **Le principe de Stealth de WireGuard :** WireGuard a été conçu pour être invisible aux attaquants. Lorsqu'il reçoit un paquet sur le port 51820, il vérifie immédiatement la signature cryptographique du paquet. Si le paquet ne provient pas d'un "Peer" explicitement configuré et connu (dont la clé publique est dans le fichier de configuration), **WireGuard rejette le paquet silencieusement (Drop)**. Il ne renvoie aucune erreur ICMP, aucun message de refus. Pour toute personne non authentifiée, le serveur semble tout simplement éteint ou inexistant sur ce port.

## Q5.

*Comparez le temps de mise en place du tunnel WireGuard vs OpenVPN. Pourquoi WireGuard est-il plus rapide à établir la connexion ?*

**Comparaison du temps :** La mise en place d'un tunnel WireGuard est quasi-instantanée (quelques millisecondes), contre plusieurs secondes pour OpenVPN.

**Pourquoi WireGuard est-il plus rapide ?** Il y a deux raisons principales :

1. **Pas de PKI (Pas de certificats) :** OpenVPN doit vérifier la validité du certificat client, du certificat serveur, vérifier qu'ils sont signés par la bonne Autorité de Certification, puis négocier une suite de chiffrement. WireGuard saute toutes ces étapes : les clés publiques (Curve25519) sont déjà pré-échangées dans les fichiers de configuration.
2. **Handshake en 1-RTT (Round Trip Time) :** La poignée de main cryptographique de WireGuard est extrêmement optimisée. Il suffit d'un seul aller-retour réseau pour que le client et le serveur soient d'accord sur la clé de session et commencent à transmettre des données utiles. L'échange TLS d'OpenVPN nécessite de multiples allers-retours avant que le premier paquet utile ne puisse passer.

## Comparaison des fichiers de configurations finaux des serveurs:

### OpenVPN:

```
GNU nano 7.2 /etc/openvpn/server/server.conf
Fichier server.conf sur S4
port 1194
proto udp
dev tun

Chemins vers les certificats et clés fournis
ca ca.crt
cert OpenVPN.crt
key OpenVPN.key
dh dh.pem
tls-auth ta.key 0

Paramètres du réseau VPN (le tunnel)
server 10.8.0.0 255.255.255.0
push "route 192.168.30.0 255.255.255.0"
push "redirect-gateway def1"
push "dhcp-option DNS 192.168.30.5"
#log
keepalive 10 120
cipher AES-256-CBC
persist-key
persist-tun
status openvpn-status.log
verb 3
```

### Wireguard

```
GNU nano 7.2 /etc/wireguard/wg0.conf
[Interface]
Address = 10.10.0.1/24
ListenPort = 51820
PrivateKey = [REDACTED]HQ=

[Peer]
PublicKey = FwJAIYfqiC0pRy+S1K/Uv8hpsGjgmeGXXbzB/xh4Kng=
AllowedIPs = 10.10.0.2/32
```

# Conclusion

## ***Deux visions de la sécurité réseau :***

Ce projet nous a permis de construire et de sécuriser notre propre infrastructure réseau de bout en bout. Mais au-delà de la simple mise en place de tunnels, il nous a surtout offert un poste d'observation privilégié pour comparer deux philosophies de la sécurité informatique : la méthode classique et l'approche moderne.

OpenVPN, le colosse de l'industrie : Lors de sa configuration, nous avons pu constater la lourdeur et la rigueur qu'exige ce protocole historique. Pour établir la confiance, nous avons dû déployer une véritable bureaucratie numérique (la PKI), avec ses autorités de certification, ses clés privées et ses signatures complexes. De plus, sa gestion du trafic nécessite d'intervenir "manuellement" sur le système, en poussant des routes et en gérant des traductions d'adresses (NAT) de manière explicite. C'est une technologie robuste et hautement personnalisable, mais qui pardonne peu les erreurs de configuration.

WireGuard, la révolution par la simplicité : À l'inverse, le passage à WireGuard a été une révélation d'efficacité. Fini la bureaucratie des certificats : ici, tout repose sur un simple échange de deux clés (publique et privée), exactement comme pour un accès SSH. En quelques lignes de configuration, le tunnel était opérationnel. Nous avons également pu observer son incroyable furtivité (invisible aux scanners de ports) et sa capacité à rétablir la connexion instantanément après une coupure réseau, là où OpenVPN aurait dû tout renégocier péniblement.

## ***Le verdict de l'ingénieur :***

Aujourd'hui, OpenVPN reste indispensable car il est installé partout et s'adapte à tous les environnements d'entreprise (même les plus anciens). Cependant, WireGuard représente sans conteste l'avenir du VPN : en éliminant la complexité, il réduit les risques de failles humaines, offre des performances inégalées et rend l'administration réseau infiniment plus agréable. Ce TP démontre parfaitement qu'en informatique, la sécurité maximale se trouve souvent dans la simplicité absolue.



# Ressources

## Configuration OpenVPN

- Support de cours et TP
- <https://doc.ubuntu-fr.org/openvpn>
- <https://wiki.debian.org/OpenVPN>

## Configuration Wireguard

- Support de cours et TP
- <https://www.wireguard.com/>
- <https://www.it-connect.fr/mise-en-place-de-wireguard-vpn-sur-debian-11/>